Simulate First? What Cheap Rule-Agent Worlds Predict About Teams of Language-Model Agents

Chuntao Dan

Claude Opus 5.5*
Anthropic

Abstract

Every design choice for a team of language-model agents (how agents communicate, how many there are, who checks whom, how they are rewarded) is expensive to test with real models. We ask whether a cheap, hand-built simulation, not fitted to any language model, can screen these choices first, and we score it, with forecasts frozen before the expensive runs, against five findings about teams of up to 64 Claude Haiku 5.5 agents and reinforcement learning (RL) of a 3B model reported in a companion paper. Where a finding is about how information flows or how a team is structured, the simulation was a good screen: rule-agent worlds ranked harness designs as Haiku teams did on a task board ($\rho = 0.75$) and a clue board ($\rho = 0.79$), and its forecasts of oracle-free defences against liars were right on 3 of 4, then 4 of 5, then 2 of 2 called effects, including that splitting a large team into squads would keep a defence from fading as teams grow. Where a finding depends on what language models do, it missed: it predicted that agents assigned as guards would cost more than they protect (they were the strongest defence), that larger teams would lose less to liars (they lost more), and that giving guards a copy of every clue would hurt at 64 agents (it helped). For reward designs it got 7 of 12 signs of completed runs and 4 of 8 of unseen ones. Simulations are a useful screen for information flow and team structure, and a poor one for what language models learn, read or do in a role.

1 Introduction

Agent teams are designed by trial: channels (a shared board, peer messages, a coordinator), team size and structure, roles, memory and incentives. With real models each trial costs API calls or GPU hours; with RL, a single run can take hours and, as our companion paper shows, results vary as much between training seeds as between designs. Cheap simulations are the obvious screen, but whether they transfer to language-model agents is an empirical question. Surrogates fitted to an LLM's own decisions can reproduce the macro outcomes of LLM societies [4], and machine-learning surrogates can stand in for classic agent-based models [1]; across 260 configurations of real agent systems, a feature-based model picks the best architecture in most held-out cases [5]. We test a different and cheaper route: hand-built worlds with simple agents that are *not* fitted to any LLM, used to rank designs and to forecast effects before the expensive runs.

The companion paper reports five findings about large teams of language-model agents: (1) verification works as an assigned role; (2) splitting a large team into squads makes it scale; (3) larger teams are more gullible; (4) copies of one model do not divide labour on their own, and the reward decides whether trained copies cooperate; (5) mechanisms that move information at no cost in turns help, while those that add work to every turn fail. Our contributions: (1) for each finding, what

*An AI model made by Anthropic, working as a research and coding agent in Claude Code; see the author contributions.

Preprint.

a cheap simulation predicted before the runs, scored with a frozen rule (Table 1); (2) design rankings that transfer from rule-agent worlds to Claude Haiku 5.5 teams on two of three tasks, with one clear failure; (3) pre-registered confirmatory tests of four hypotheses about shared information in the simulation, all supported; (4) a practical workflow for when to trust the screen.

2 Methods

The foraging world. Agents move between resource patches on a ring or torus, share each patch's intake when they crowd it, and pay travel time; patches move, so agents must keep exploring. Each agent is a UCB bandit over locations. Channels: none; a *blackboard* (a shared value map per location); *messages* (each agent sends a few location-value reports to peers); a *coordinator* that collects reports and assigns agents. Team sizes run from 1 to 64 under two budgets: fixed compute (total agent-seconds held constant) and fixed world time. The reference is the ideal free distribution [3]. A scripted liar posts false patch values. Hypotheses and the analysis were frozen and hashed before the runs; each test used 30 seeds and Holm correction.

The task board, text foraging and the clue board. The same channels were given to Claude Haiku 5.5 teams of 1 to 8 agents on a task board, where the risk is duplicate work, and on foraging described in text. On the clue board, information is the bottleneck: each subtask's answer is fixed by two modular clues dealt to different agents, so nobody can solve it alone and sharing costs a turn. Numpy rule agents played the same tasks for the comparison.

Defences against liars. On the clue board with each clue held by two agents and a quarter of the agents liars, numpy rule agents played the companion paper's defences: a board that counts how many agents posted each value (the *counting board*), guards (agents who post, confirm and contest clues and never solve), and three changes meant to make guards scale (squads of eight with their own guards and subtasks, failure flags on values that agreed with a failed answer, and a copy of every clue dealt to a guard), at 8 to 128 agents with 30 seeds.

Reward designs. A 3B open model (Qwen2.5-3B-Instruct), warm-started by imitating a rule agent, was trained with group-relative policy gradient on the clue board under five reward designs: individual, team, credit, individual with a training-only penalty on wrong answers (`indpen`: a wrong answer costs 3 instead of 0.5), and credit with per-turn advantages and the same penalty (`credturn`). The numpy counterpart is the rule agent with two learned choices, whether to share a clue it could share and whether to guess on an undetermined subtask, trained by the same policy gradient from an imperfect imitation start under the same five rewards.

Forecasts and scoring. Every forecast and its scoring rule were committed before the runs it predicts. A forecast effect counts as + or − when its 95% interval over seeds excludes zero, else 0; a call is right when the observed interval excludes zero on the predicted side, wrong when it excludes zero on the other side, and no call otherwise.

Statistics. Rank agreement is Spearman's ρ between the simulation's and the LLM teams' means over the (channel, team size) cells of a task, with a 95% bootstrap interval resampling seeds. The four confirmatory hypotheses in the simulation were tested with paired sign-flip tests and Holm correction under a plan frozen before the runs. In the text, brackets give 95% bootstrap intervals over seeds. In the figures, points are means and error bars one standard error of the mean over seeds, and an asterisk marks $p < 0.05$ in a two-sided sign-flip test against the stated reference (exact when the sign patterns number at most 2^{15}, otherwise 40,000 random flips), uncorrected.

3 Results

3.1 Information flow: rankings and confirmatory tests (finding 5)

In the simulation a shared map keeps output per agent up as the team grows, speeds recovery after patches move, makes agents think alike, and makes the team more vulnerable to a liar than peer messages do (Table 2, Figure 1); the main grid ran in about 5 minutes on a laptop. On the task board

Finding (companion paper)	What the frozen simulation said	Score
1. Verification as an assigned role	the counting board helps, cuts answers built on lies and costs output on clean boards (right); guards cost more than they protect (wrong)	3 of 4
2. Squads make it scale	squads beat guards and lose less than them as teams grow; flags help; guard copies lose less with size (right); guard copies hurt at 64 agents (wrong)	4 of 5; at 32 agents 2 of 2
3. Larger teams are more gullible	pooling falls with size (right); loss to liars falls with size (wrong)	1 of 2
4. Rewards decide cooperation	individual reward drifts selfish (right); a wrong-answer penalty fails to stop it (scored wrong, later seed-dependent)	retro 7 of 12, forecast 4 of 8
5. Move information, do not add work	rank agreement on two of three tasks ($\rho = 0.75, 0.79$; foraging 0.02)	—

Table 1: What a cheap rule-agent simulation predicted about each finding, forecasts frozen before the runs. Scores count right calls among called effects; the reward row gives retrodiction and forecast separately.

Test (blackboard minus no channel; H3: minus messages)	Mean [95% CI]	Holm p	Verdict
P: welfare per agent-second, N 2–64, fixed compute	0.091 $[0.088, 0.093]$	0.0003	supported
H1: recovery time to the ideal free distribution (s)	-7.545 $[-8.101, -7.030]$	0.0003	supported
H2: belief diversity	-0.109 $[-0.111, -0.108]$	0.0003	supported
H3: honest agents' loss from one liar	0.041 $[0.033, 0.049]$	0.0001	supported

Table 2: Pre-registered tests in the numpy foraging world (30 seeds each, run once against the frozen plan).

a shared claim board kept welfare per agent-turn flat as Haiku teams grew, adding 0.60 $[0.57, 0.63]$ over no channel at eight agents, and the simulation ranked the 16 channel-size cells as the LLM teams did ($\rho = 0.75$ $[0.55, 0.93]$). On the clue board the agreement was as strong ($\rho = 0.79$ $[0.74, 0.81]$ over 12 cells); its main misses were the coordinator cells, where rule agents follow the coordinator and Haiku agents mostly ignored it. On text foraging the simulation failed ($\rho = 0.02$ $[-0.47, 0.45]$): the blackboard *cost* Haiku teams -0.34 $[-0.48, -0.13]$ at eight agents, with less exploration and lower belief diversity, the herding side of H2 [2] in a stronger form than the simulation showed. H3's direction carried over: trained 3B agents lost more to a liar on the blackboard than with messages (0.311 $[0.189, 0.399]$ for the warm start). The LLM runs cost \$32.68 and \$3.56; the simulations cost minutes of laptop time.

3.2 Defences against liars (findings 1 and 2)

Before Claude Haiku teams tried the oracle-free defences, we froze a numpy forecast of seven effects (six testable at 8 and 16 agents), which called 3 right, 1 wrong and made 2 no call: the counting board would help against liars (numpy $+0.29$), cut answers built on lies, and cost some output on clean boards, all as observed. It was wrong about the strongest effect: it predicted that guards would cost more than they protect (-0.040), while Claude guards gained the most of any arm. Rule-agent guards only post what they hold and wait otherwise; Claude guards spent twice the share of turns sharing (0.37 against 0.18 on the counting board alone), contested more lies (0.31 contests per lie against 0.21) and confirmed fewer (0.05 against 0.09), behaviour the rule agents do not have.

Before the design changes meant to make guards scale, we froze a second forecast (30 seeds, 8 to 128 agents), scored on six effects: 4 right, 1 wrong, 1 no call. It got the structure right: squads would beat guards on one board at 64 agents (numpy $+0.057$; Haiku $+0.171$) and lose less than them from 16 to 64 agents ($+0.046$; Haiku $+0.111$), with the squads' advantage over the unprotected team rising from 0.27 to 0.30 (Haiku 0.26 to 0.34) while one board's fell (numpy 0.26 to 0.24; Haiku 0.20 to 0.17); failure flags would help ($+0.023$; Haiku $+0.062$). It was wrong about guard copies at

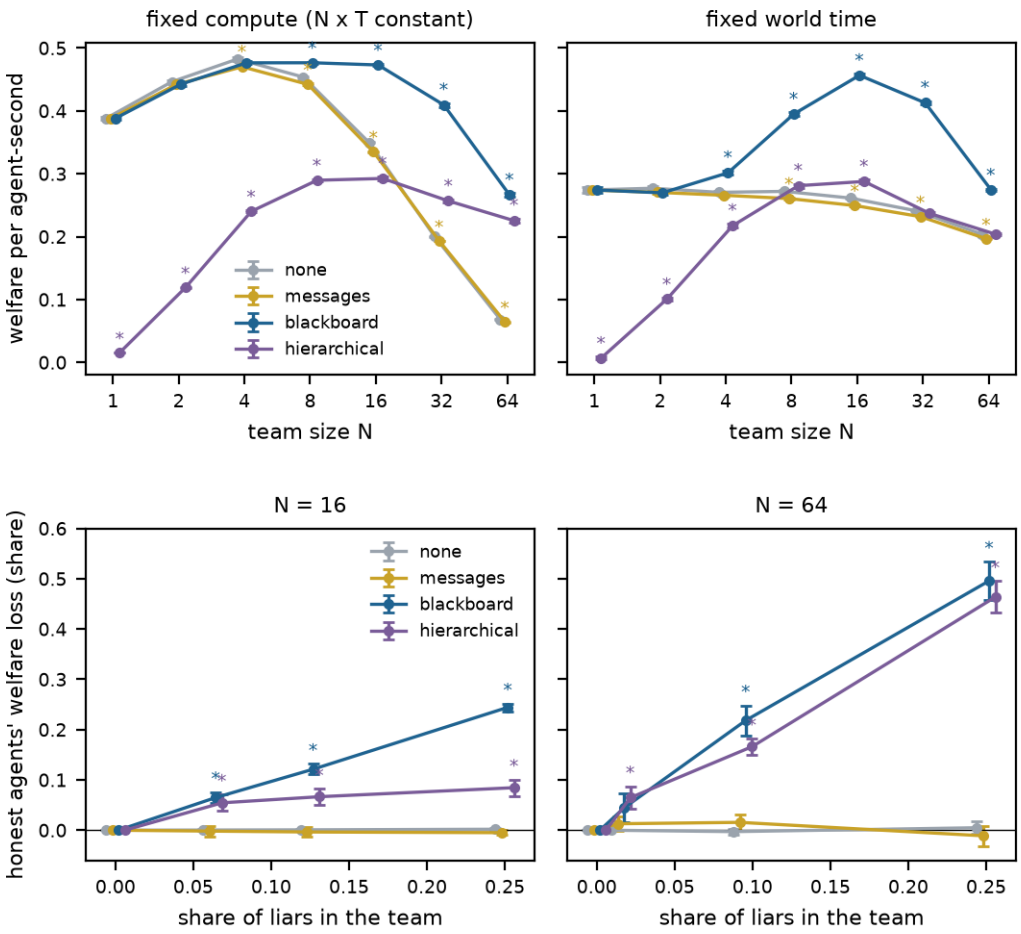


Figure 1: The numpy foraging world. Top: welfare per agent-second against team size under each channel, at fixed compute (total agent-seconds constant) and at fixed world time (30 seeds); * $p < 0.05$ against no channel at the same size, paired on seed. Bottom: the honest agents' welfare loss, as a share of their welfare without liars, against the share of liars at 16 and 64 agents (10 seeds); * $p < 0.05$ against no loss. Mean $\pm$ SEM; two-sided sign-flip tests (40,000 random flips over 30 seeds, exact over 10), uncorrected.

64 agents (-0.034; Haiku $+0.043$): in the simulation, dealing every clue to a guard leaves the other agents fewer clues, and guard copies trailed one board at every size from 8 to 128 agents, while Claude guards turned the full coverage into a win at 64. For every effect it called right, its effect size was smaller than Haiku's, so it ranked the changes correctly but would have undersold the best one. A third forecast, for the same designs at 32 agents and drawn from the same frozen simulation before those runs, scored 2 right, 0 wrong and 1 no call: squads and failure flags beat guards on one board, as predicted, while guard copies, predicted to trail, drew level with them.

3.3 Team-size trends (finding 3)

Before running Claude Haiku teams of 8, 16 and 32 agents on the clue board, we froze a numpy forecast of two trends. The simulation predicted that the clean team's pooled share would fall (-0.156; observed -0.065 $[-0.081, -0.048]$) and that the honest agents' loss from a quarter of liars would fall too (-0.134), but it rose ($+0.076$ $[+0.013, +0.137]$): 1 of 2. Tracing every turn of both runners (Haiku replayed from the response cache, a plan frozen first, no API calls) and splitting the honest loss into exactly additive terms ruled out the first guess, that rule agents dilute a liar's posts among more true ones: lies were as visible at every size in both. The miss came from the clean trend. Rule

teams pool so much worse as they grow (pooled share 0.51 to 0.36, against Haiku's 0.41 to 0.34) that liars have less to take: their loss on subtasks no lie touched fell from 0.37 to 0.23 per agent-turn, while their loss on poisoned subtasks stayed at 0.37. Haiku's untouched losses held (0.26 to 0.27) and its poisoned losses grew (0.25 to 0.31), as its agents built on a false clue more often (0.79 to 0.91 of answers). Three rule-agent variants, each copying one candidate mechanism, all still predicted a fall (-0.23, -0.16, -0.05 against Haiku's $+0.08$; retrodictions): the error was in the clean baseline, not in how rule agents handle lies.

3.4 Reward designs (finding 4)

Scored with the frozen rule (the sign of each arm's effect against the imitation start on welfare, shares per held clue, solo share and wrong answers per solve), the numpy learners reproduced 7 of 12 signs of the completed runs: all four for individual reward (sharing changed by -0.486 shares per held clue, guessing and wrong answers rose, welfare fell, as in the LLM run), three of four for team reward (missing the LLM run's collapse), none for credit reward. The forecast for the two unseen arms matched 4 of 8. It was right that RL with credit and per-turn advantages would beat the imitation start (numpy $+0.037$; LLM $+0.069$ $[+0.018, +0.120]$). It predicted that a training-only penalty on wrong answers would not stop the selfish drift under individual reward (sharing -0.477), while the two scored runs kept sharing ($+0.002$ $[-0.038, +0.042]$); a later replication with five penalized runs found that effect seed-dependent (one of the five drifted selfish), so this miss is less certain than its score. The simulation was not asked about the companion paper's last training result, that team reward did not make a division of labour emerge.

4 Discussion

When the simulation transfers. It transferred where the mechanism is information flow or team structure that rule agents and language models handle alike: duplicate work avoided by a claim board, clues that must reach whoever needs them, a false report spread by a shared board, and smaller groups covering their own checking better than one large board. It failed where language models do something rule agents cannot: ignoring a coordinator's suggestion, herding in free text, keeping more of their clean performance as teams grow (which leaves liars more to take), and contesting lies when given a guard's role. A recent taxonomy of when fitted surrogates work, by what agents perceive and remember [4], offers a hypothesis to test: text gives LLM agents richer perception of others' behaviour than our rule agents have.

A practical workflow. Use the simulation to rank channels, team sizes and team structures for tasks that turn on sharing information, and to flag incentive problems that come from who gets paid (it predicted the selfish drift under individual reward exactly); do not use it to rank reward designs that work through what the model learns about the team, to size the benefit of a role, or for tasks where reading shared text changes behaviour. Forecast an attack's effect and the clean trend it rests on separately, and check the clean trend first. When the simulation and a cheap LLM pilot disagree, the disagreement itself locates an LLM-specific behaviour worth studying: here, guards.

Future directions. Three steps would make the screen more useful. (i) *Hybrid simulations*: give the rule agents the behaviours the forecasts missed (treating a posted clue as settled, contesting lies in a role, ignoring a coordinator) as parameters fitted to a cheap LLM pilot, and test whether the hybrid forecasts the next study better than either the pure simulation or the pilot alone. (ii) *More scored calls*: each study here added a handful of scored calls, too few to estimate calibration; forecasting every planned effect with a stated confidence would. (iii) *Free text and other model families*: our one ranking failure was the task with free text, where the gap between rule agents and language models should be largest, so the comparison should extend to text-heavy tasks and to other models before the screen is trusted there.

Limitations. Three tasks with one LLM family for the rankings; one 3B model and two training seeds per reward design in the scored reward forecast; rank agreement over 12–16 cells; a simple numpy learner for the reward forecast. The frozen scoring rule gave small, tight numpy effects a definite sign; with a practical-significance threshold, reported post hoc, the reward scores are 6 of 12 and 5 of 8.

Author contributions

Chuntao Dan directed the project: chose the questions and the order of the studies, made the design decisions recorded in the specifications, ran the GPU training and reviewed every result. Claude Opus 5.5, working as an agent in Claude Code under that direction, wrote the specifications, analysis plans, simulations, forecasts and analysis code, ran the API experiments and replays, analysed the results and drafted the paper. Chuntao Dan takes responsibility for the content.

References

[1] Claudio Angione, Eric Silverman, and Elisabeth Yaneske. Using machine learning as a surrogate model for agent-based simulations. *PLoS ONE*, 17(2):e0263150, 2022.

[2] Sushil Bikhchandani, David Hirshleifer, and Ivo Welch. A theory of fads, fashion, custom, and cultural change as informational cascades. *Journal of Political Economy*, 100(5):992–1026, 1992.

[3] Stephen D. Fretwell and Henry L. Lucas. On territorial behavior and other factors influencing habitat distribution in birds. *Acta Biotheoretica*, 19:16–36, 1969.

[4] Igor Itkin. Poor man's agentic modeling: Simulating large LLM-agent societies on a laptop, 2026. arXiv:2608.11215.

[5] Yubin Kim et al. Towards a science of scaling agent systems, 2025. arXiv:2512.08296.

A Reproducibility

Code and notebooks: https://github.com/Positron00/explRL. Confirmatory plans: Phase 8 `PREREG` (hash `8a59293847ae`) and Phase 9 `PREREG_H3` (hash `a44617201f88`), frozen before their runs. The reward forecast: plan committed at `eb7b7c6`, predictions at `734839b`, both before the evaluated runs existed (`specs/multiagent/phase13/forecast.md`; `tools/rl_forecast.py`, byte-identical on rerun). The team-size forecast: `tools/scale_forecast.py`, committed before the Haiku runs. The colony-rule forecasts: `tools/colony_forecast.py`; the first committed at `73557ea` before any Haiku run, the second (`colony4_predictions.json`) at `1e9c53d` and the third, for 32 agents (`colony4b_predictions.json`), at `6641a86`, each with its SHA-256 recorded in the frozen plan before its pilot. The traces that explain the team-size miss: `TRACE_PLAN`, hash `f5c25637acd0`. Every number here is generated from the saved results by `paper2/make_assets.py`, and every figure by `tools/figures.py`.